CCA 3 – COMPUTER VISION

NAME – Bedge Aishwarya Kailas

PRN – 1272250514

CLASS – S.Y MCA DIVI – C (Batch 2)

GIT HUB LINK : - https://github.com/Aish134/computer-vision-cca3.git

1. Introduction

Computer Vision is a field of Artificial Intelligence that enables a computer to process and understand visual information such as photographs and videos. A computer does not understand an image in the same way a human does. It receives an image as a collection of numerical pixel values. Computer vision techniques process these values to improve image quality, identify important regions, describe visual patterns, detect objects and estimate movement.

The purpose of this assignment is to implement and understand several important computer vision operations using Python and OpenCV. The project is divided into four main tasks. The first task studies image restoration and segmentation. The second task extracts visual features and matches them between photographs. The third task detects faces and objects. The fourth task analyses movement in a video using three different techniques.

2. Problem Statement

Digital images may contain unwanted noise, unclear boundaries or complex backgrounds. It can be difficult to identify useful information directly from a raw image. Similarly, a computer needs suitable features and detection algorithms to recognise objects, while video analysis requires techniques to identify changes and movement between frames.

The problem addressed by this project is to apply suitable computer vision algorithms to image and video inputs, observe their outputs, and understand the strengths and limitations of each method. The project uses a portrait photograph, a second photograph of the same subject/scene for feature matching, a video, and a pretrained YOLO model.

3. Objectives

- Generate Gaussian and salt-and-pepper noise and understand how they affect image appearance.
- Apply Gaussian and median filters to reduce the corresponding noise types.
- Evaluate restoration quality with PSNR and SSIM instead of relying only on visual inspection.
- Perform thresholding, morphological processing, edge detection and watershed segmentation.
- Extract HOG features to describe local gradient and shape information.
- Use SIFT keypoints and descriptors to find matching visual points in two photographs.
- Repeat feature matching after resizing an image to study scale changes.
- Detect face-like regions using Haar Cascade and general object classes using YOLO.
- Estimate video motion using frame differencing, MOG2 background subtraction and Lucas–Kanade optical flow.
- Compare the methods and explain their results and limitations.

4. Tools, Technologies and Input Files

Tool / input	Role in this project
Python	Implements the processing pipeline and connects the computer vision libraries.
Jupyter Notebook / Anaconda	Provides the interactive environment to execute code cells and display outputs.
OpenCV	Reads and processes images and videos; performs filtering, thresholding, morphology, edge detection, SIFT, Haar Cascade and video operations.
NumPy	Stores images as arrays and performs numerical operations, including noise generation.
Matplotlib	Displays original images, processed images, feature matches, detection boxes and motion results.
scikit-image	Provides PSNR, SSIM and HOG-related functionality.
Ultralytics YOLO	Loads the supplied YOLO11n model weights and performs object detection.
Pandas	Organises measured values and sampled video metrics in tables.
original_portrait.png	Main portrait image used for restoration, segmentation, HOG and object detection.
second_same_scene_photo.jpg	Second original photograph used for SIFT matching.
original_video.mp4	Video input used for motion estimation.
yolo11n.pt	Pretrained YOLO model weights used for object detection.

The notebook's saved setup output reports that the portrait is 1103 × 1426 pixels, the second photograph is available, and the video is available. OpenCV version 4.10.0 is reported. The video properties reported by the notebook are 3840 × 2160 pixels, 29.97 frames per second, 670 frames and 22.36 seconds.

5. Task 1 – Image Restoration and Segmentation

5.1 Understanding the input image

An image is represented in a computer as a grid of pixels. Each pixel stores intensity information. For a colour image, the pixel usually contains three colour-channel values. The notebook loads the portrait and creates a grayscale version where each pixel represents brightness rather than separate colour channels. Grayscale images are useful for operations such as thresholding, edge detection and face detection because these operations mainly examine intensity changes.

5.2 Adding Gaussian noise

Gaussian noise is random variation added to pixel values. It can represent sensor noise or unwanted fluctuations in an image. In this experiment, random values following a Gaussian distribution are added to the original portrait. The result looks grainier because pixel intensities are changed slightly throughout the image.

The notebook uses a fixed random seed so that the random noise pattern can be reproduced when the cell is run again. The implementation generates the noise in small row blocks using float32 arrays. This chunk-based approach is important because creating a full-size floating-point noise array for a high-resolution image can consume a large amount of RAM and cause a MemoryError.

5.3 Adding salt-and-pepper noise

Salt-and-pepper noise changes selected pixels to extreme values. A salt pixel becomes white, while a pepper pixel becomes black. These isolated bright and dark dots are visually different from Gaussian noise, which changes pixel values more gradually. The notebook selects a portion of pixel locations and replaces them with black or white values.

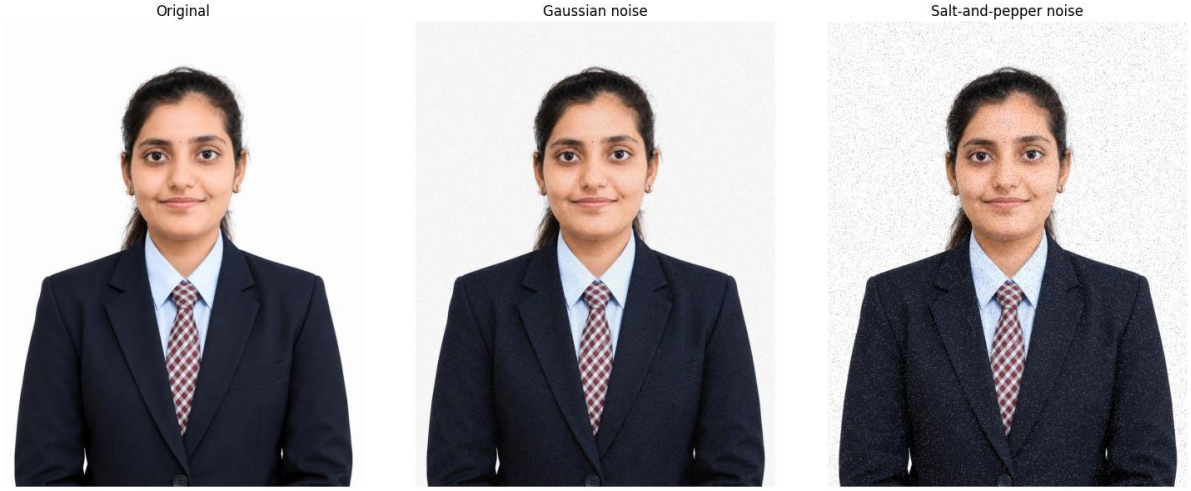


Figure 1. Original portrait, Gaussian-noisy image and salt-and-pepper-noisy image.

5.4 Restoring the Gaussian-noisy image

The notebook applies Gaussian blur using a 5 × 5 kernel. A Gaussian filter calculates a weighted average of nearby pixels. Pixels closer to the centre of the filter contribute more strongly than pixels farther away. This averaging smooths small random intensity variations and reduces Gaussian noise.

The filter has a trade-off: strong smoothing may remove some fine details and make edges appear softer. Therefore, the purpose is not simply to blur the image as much as possible, but to reduce noise while retaining important visual structure.

5.5 Restoring the salt-and-pepper-noisy image

A median filter with a 5 × 5 neighbourhood is used for salt-and-pepper noise. Instead of averaging the neighbouring pixel values, the median filter sorts the values and selects the middle value. An isolated black or white pixel is often very different from its neighbours, so the median operation can remove it without spreading its extreme value to surrounding pixels.

Median filtering is therefore commonly suitable for impulse noise. It can preserve edges better than a simple averaging filter, although the result still depends on the kernel size.

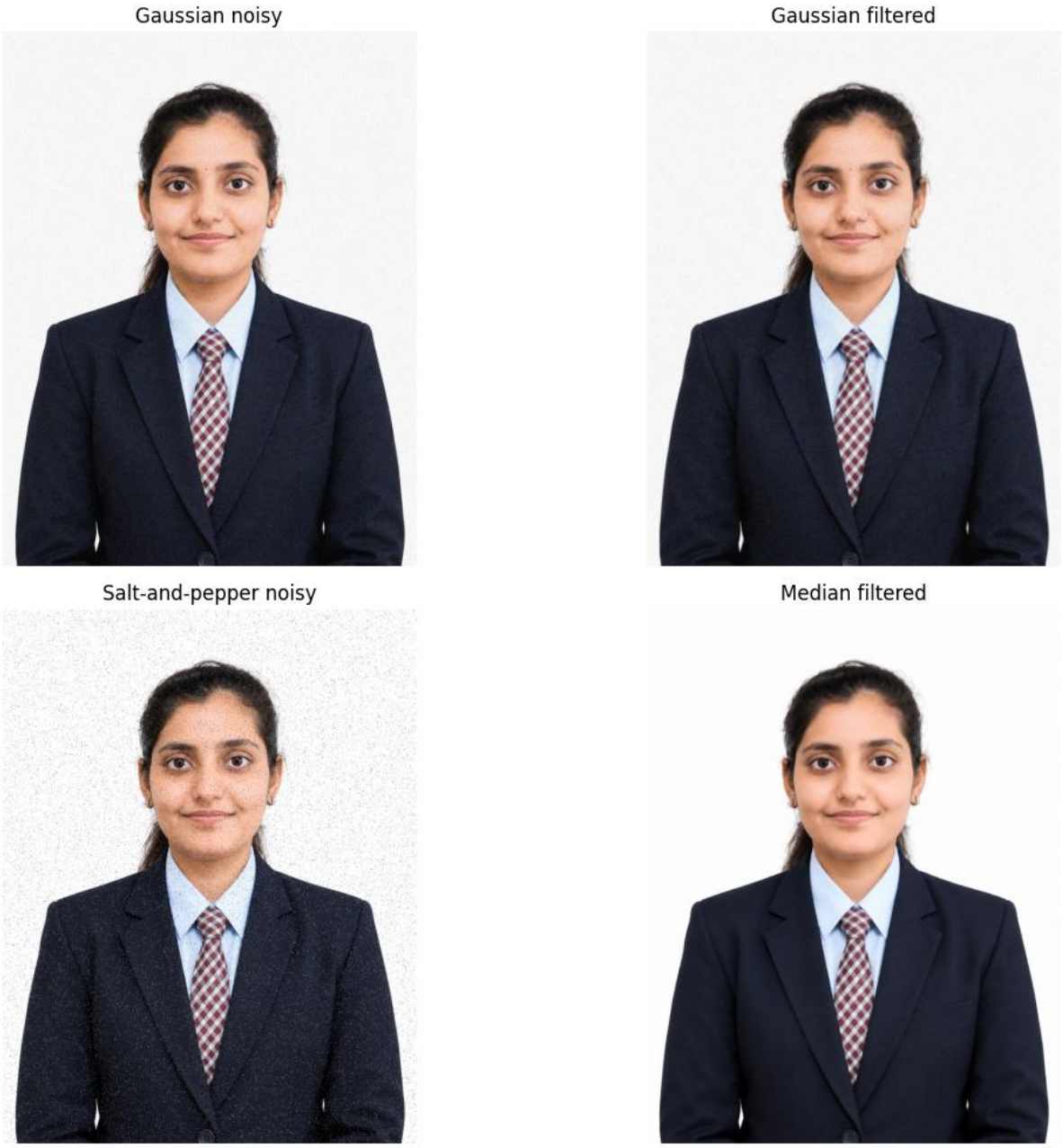


Figure 2. Gaussian filtering and median filtering results.

5.6 Measuring image quality with PSNR and SSIM

Visual inspection is useful, but it can be subjective. The notebook therefore compares each processed image with the original reference image using two numerical measures.

Peak Signal-to-Noise Ratio (PSNR) expresses the difference between a reference image and a processed image in decibels. A higher PSNR generally means the processed image has a smaller pixel-wise reconstruction error. It is based on the Mean Squared Error (MSE), which averages the squared differences between corresponding pixel values.

Structural Similarity Index (SSIM) compares local luminance, contrast and structural patterns. Its value is generally interpreted as better similarity when it is closer to 1. SSIM can be more aligned with perceived structural quality than a simple pixel error measure.

Image condition	PSNR (dB)	SSIM
Gaussian noisy	24.3498	0.3133
Gaussian filtered	30.5018	0.8080
Salt-and-pepper noisy	18.2351	0.4299
Median filtered	35.5865	0.9571

Interpretation: after Gaussian filtering, PSNR increases from 24.3498 dB to 30.5018 dB and SSIM increases from 0.3133 to 0.8080. This indicates that the filtered image is more similar to the original than the noisy image. For salt-and-pepper noise, median filtering increases PSNR from 18.2351 dB to 35.5865 dB and SSIM from 0.4299 to 0.9571. Among the recorded outputs, the median-filtered image has the highest similarity score.

5.7 Otsu thresholding

Thresholding converts a grayscale image into a binary image. Each pixel is assigned to one of two groups depending on whether its intensity is above or below a threshold. Otsu's method automatically chooses a threshold by analysing the image's intensity histogram. It searches for a threshold that separates the two pixel groups as effectively as possible according to the between-class variance criterion.

The result is a black-and-white mask. Thresholding is useful when the foreground and background have sufficiently different intensity values. It may be less effective when lighting is uneven or foreground and background intensities overlap.

5.8 Morphological operations

Morphological operations work on binary images and use a small shape called a structuring element or kernel. The notebook uses a 5 × 5 kernel. The operation changes the size and shape of white foreground regions.

- Erosion: removes pixels from the boundaries of white regions. It makes foreground objects smaller and can remove tiny white noise.
- Dilation: adds pixels to the boundaries of white regions. It makes foreground objects larger and can connect nearby white regions.
- Opening: erosion followed by dilation. It is useful for removing small foreground noise while attempting to retain the main object shape.
- Closing: dilation followed by erosion. It is useful for filling small holes and closing narrow gaps inside foreground regions.

5.9 Canny edge detection

Canny edge detection identifies locations where image intensity changes sharply. It first reduces noise, calculates intensity gradients, thins candidate edges through non-maximum suppression and uses two thresholds with edge tracking to retain connected edges. The output highlights boundaries rather than filling complete object regions. It can help identify object outlines, but texture and noise may also create edges.

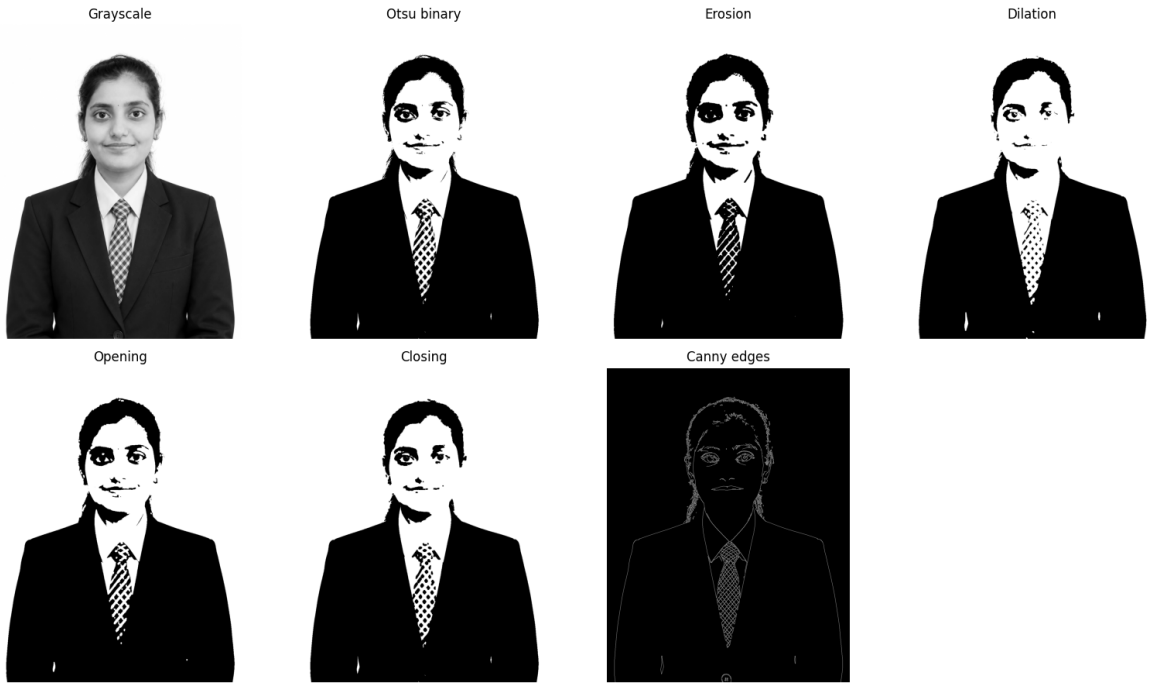


Figure 3. Otsu thresholding, morphological operations and Canny edge detection.

5.10 Watershed segmentation

Watershed is a segmentation technique used to divide an image into regions. A useful way to understand it is to imagine the grayscale image as a landscape. Low and high pixel values form valleys and hills. The algorithm uses foreground and background markers and conceptually expands regions from those markers. Where expanding regions meet, boundaries are formed.

Watershed is particularly useful when objects touch or overlap and a simple threshold cannot separate them. The quality of the result depends on suitable markers and a suitable input image. In this notebook, the watershed cell looks for an optional input named data/watershed_objects.png. If that file is not available, it uses the portrait image as a fallback and prints a note asking for a dedicated image containing multiple or touching objects. The saved execution indicates that this optional image was not present. Therefore, the watershed code is included and executes with the fallback, but the output should not be interpreted as a validated separation of touching objects.

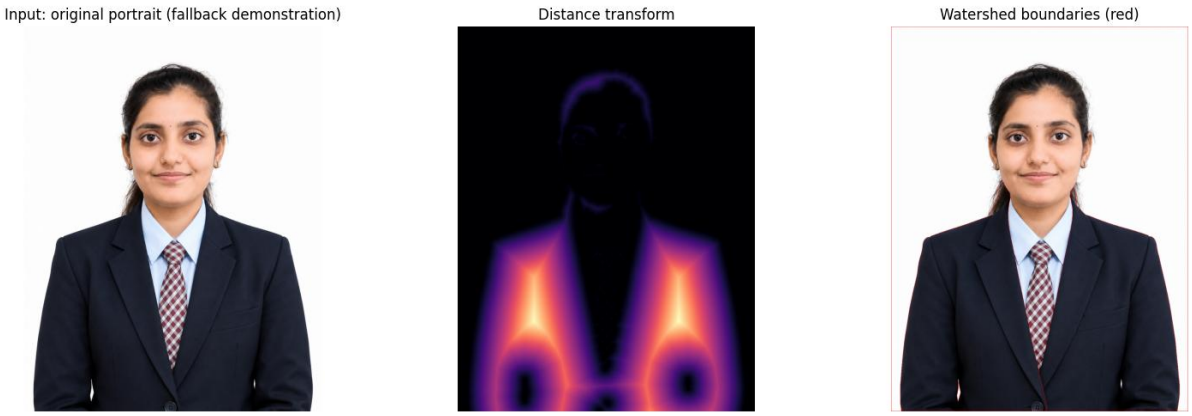


Figure 4. Watershed output produced by the notebook's available input/fallback.

5.11 Task 1 summary

Task 1 demonstrates that the correct restoration filter depends on the type of noise. Gaussian blur is used for Gaussian noise, while median filtering is effective for salt-and-pepper noise. PSNR and SSIM provide quantitative support for the visual comparison. Thresholding and morphology create and refine binary regions, Canny identifies edges, and watershed demonstrates marker-based region separation.

6. Task 2 – Feature Extraction and Image Matching

6.1 What is feature extraction?

Feature extraction converts an image into a representation that describes useful visual information. Instead of comparing every pixel directly, an algorithm can examine patterns such as edges, gradients, corners and local texture. These descriptions can be used to compare images, locate repeated patterns or track objects.

6.2 Histogram of Oriented Gradients (HOG)

HOG stands for Histogram of Oriented Gradients. It describes the shape and appearance of an image by measuring the directions of intensity changes. An edge is a location where brightness changes. The direction of this change is called the gradient orientation.

The image is divided into small cells. For each cell, HOG calculates a histogram that counts gradient directions. Nearby cells are grouped into blocks, and the histograms are normalised. Normalisation helps reduce the effect of changes in overall brightness and contrast. The resulting vector describes the local arrangement of edges.

In the notebook, the portrait is converted to grayscale and resized to 256 × 256. HOG is computed using 9 orientation bins, 16 × 16 pixels per cell, 2 × 2 cells per block and L2-Hys normalisation. The saved output reports a descriptor length of 8,100 values. HOG describes gradient structure; it does not independently decide that the image contains a particular semantic object.

Portrait input



Portrait HOG visualization

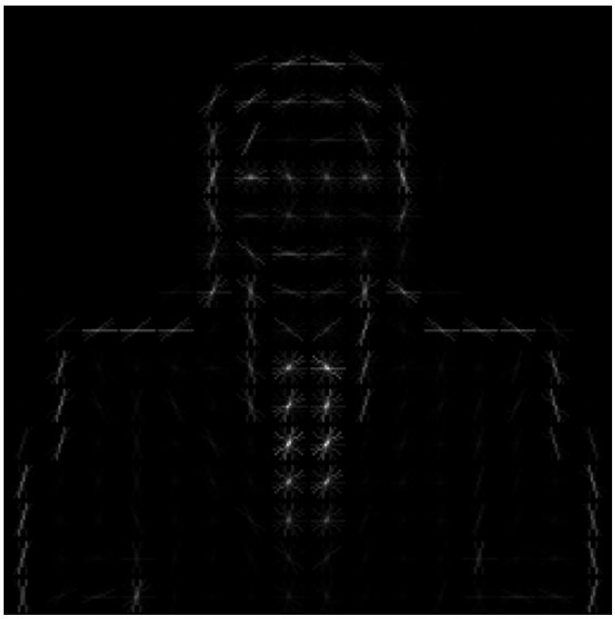


Figure 5. HOG visualisation and feature descriptor output.

6.3 SIFT keypoint detection

SIFT stands for Scale-Invariant Feature Transform. It finds distinctive points in an image that can often be recognised even when the image is resized or rotated. These points are called keypoints. Examples include corners, textured regions and other local patterns that are visually distinctive.

SIFT searches for candidate points across different image scales. It selects stable keypoints and calculates a descriptor for each one. A descriptor is a numerical representation of the local image region around a keypoint. Two descriptors can then be compared to determine whether they may represent the same visual feature in two photographs.

6.4 Matching SIFT features between two original photographs

The assignment's two-photo experiment uses the original portrait as image A and the second original photograph as image B. The second photograph should show the same object/person or scene so that some visual features are expected to appear in both images. The notebook detects keypoints and descriptors in both images and compares the descriptors using a brute-force matcher with L2 distance.

The notebook applies Lowe's ratio test. For each descriptor, the two nearest candidate matches are considered. A match is retained when the best distance is sufficiently smaller than the second-best distance. The ratio threshold used is 0.75. This helps remove ambiguous matches that may have several similarly close alternatives.

SIFT measurement	**Recorded value**
Keypoints in image A	726
Keypoints in image B	489
Good matches after ratio test	74

Interpretation: 726 keypoints were detected in image A and 489 in image B. After applying the ratio test, 74 good matches remained. This indicates that the images share a number of local visual patterns that the SIFT descriptors can correspond. A larger number of matches does not automatically guarantee that every match is correct, so the connecting lines in the visual output should also be inspected.

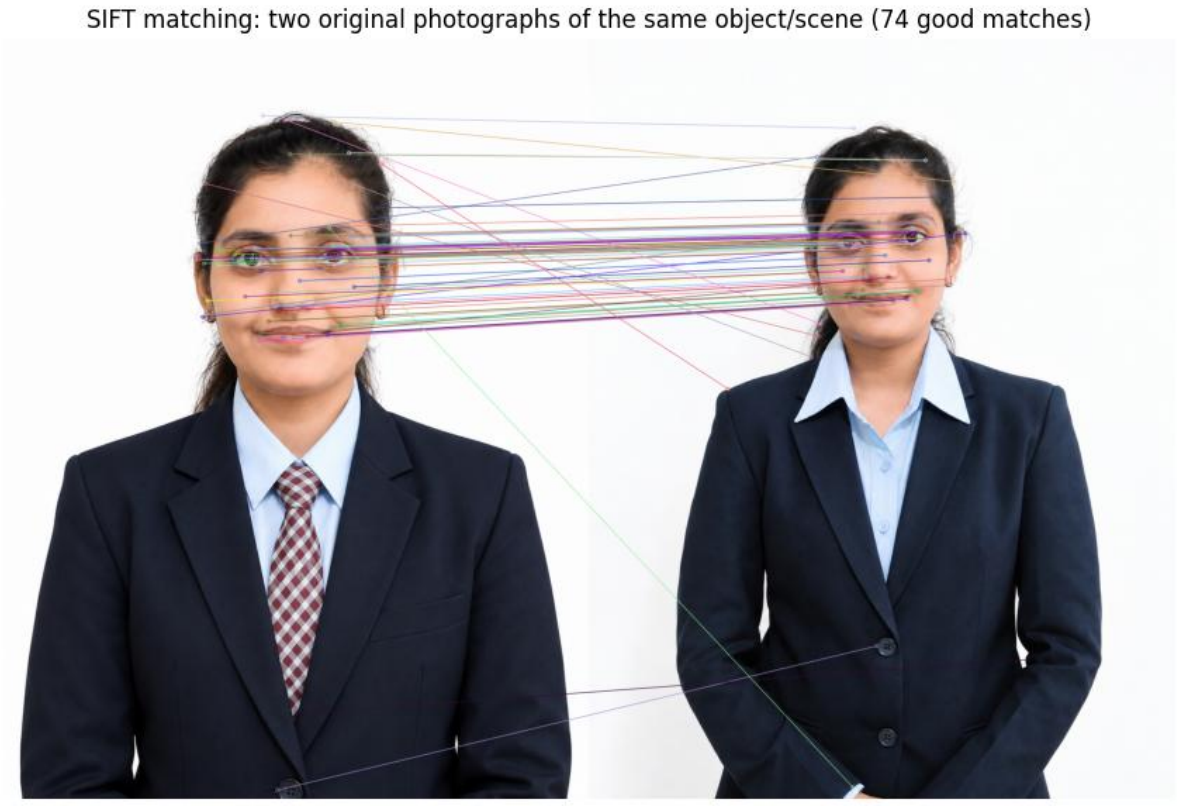


Figure 6. SIFT keypoint correspondences between the two original photographs.

6.5 SIFT matching after resizing

The notebook then resizes image B to 75 percent of its original dimensions and repeats the matching experiment. This tests scale robustness: the same visual content is represented at a smaller size. SIFT is designed to find features across different scales, although resizing can change the number of detected keypoints and the details that remain visible.

Experiment	Keypoints A	Keypoints B	Good matches
Original image A vs original image B	726	489	74
Original image B vs 75%-resized image B	489	354	263

The saved notebook output reports 354 keypoints in the resized image and 263 good matches between image B and its resized version. This demonstrates that many local features remain matchable after the scale change in this particular experiment.

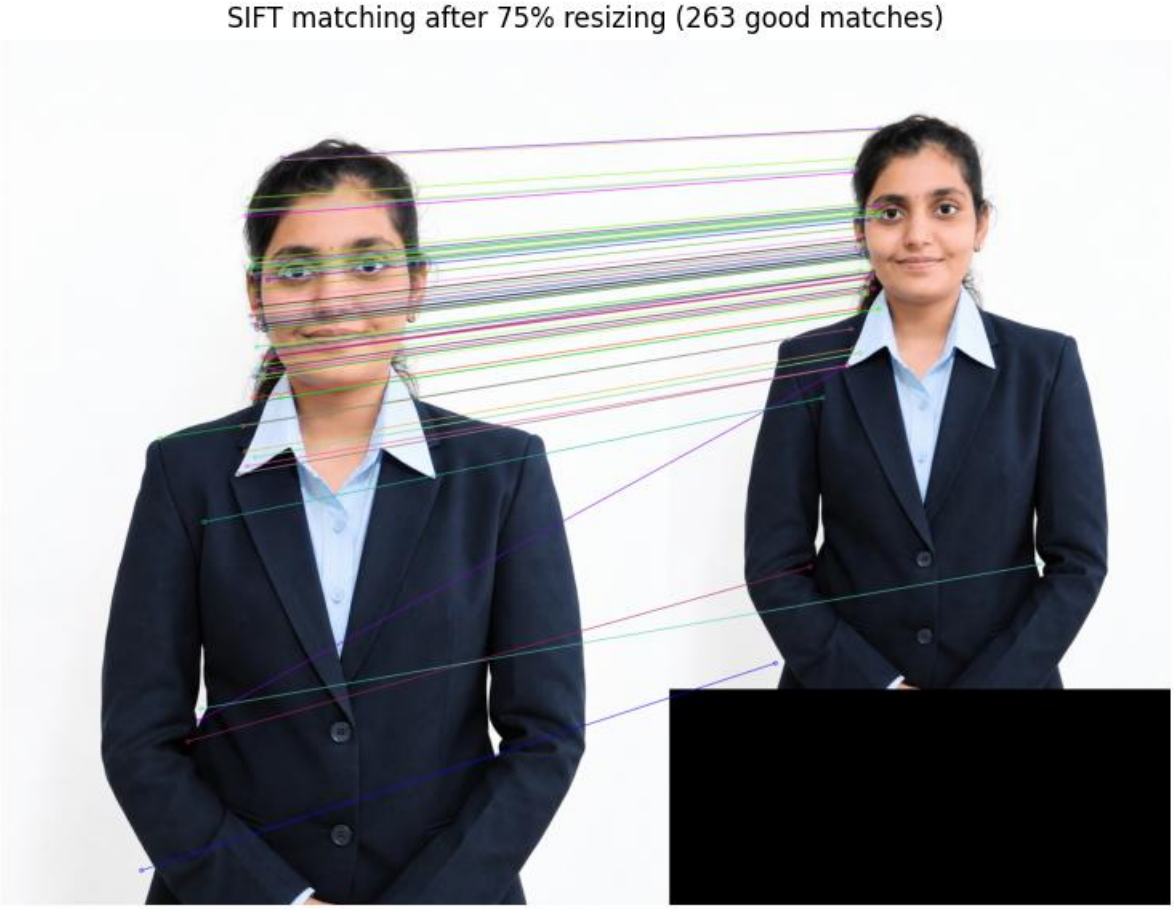


Figure 7. SIFT matching between image B and its resized version.

6.6 Task 2 summary

HOG provides a global/local gradient-based description of shape, while SIFT provides distinctive local keypoints and descriptors for correspondence matching. HOG is useful for describing edge patterns; SIFT is useful for finding similar local features across photographs and transformations. Neither feature descriptor alone is a complete object-recognition system.

7. Task 3 – Object Recognition

7.1 Object recognition overview

Object detection identifies objects in an image and usually reports their locations using rectangular bounding boxes. Some detection methods are designed for a specific target, while others can detect multiple categories. This task compares a face-oriented Haar Cascade detector with the YOLO object detection model.

7.2 Viola–Jones Haar Cascade

The Viola–Jones approach is a classical method commonly used for face detection. Haar-like features describe differences in brightness between rectangular regions. For example, the eye region may be darker than the upper

cheek region. The algorithm evaluates many such features and uses a cascade of classifiers to reject image regions that do not resemble a face.

A cascade means that the image region passes through a sequence of increasingly selective stages. Regions rejected at an early stage do not need to be processed by later stages, which makes detection relatively fast. The notebook uses OpenCV's pretrained frontal-face cascade, converts the portrait to grayscale and calls detectMultiScale. The configured scale factor is 1.1, minNeighbors is 5 and the minimum detection size is 35 × 35 pixels.

The saved output reports two Haar detections. These are face-like regions identified by the classifier. Haar Cascade can be affected by head rotation, unusual lighting, occlusion, image quality and background patterns.

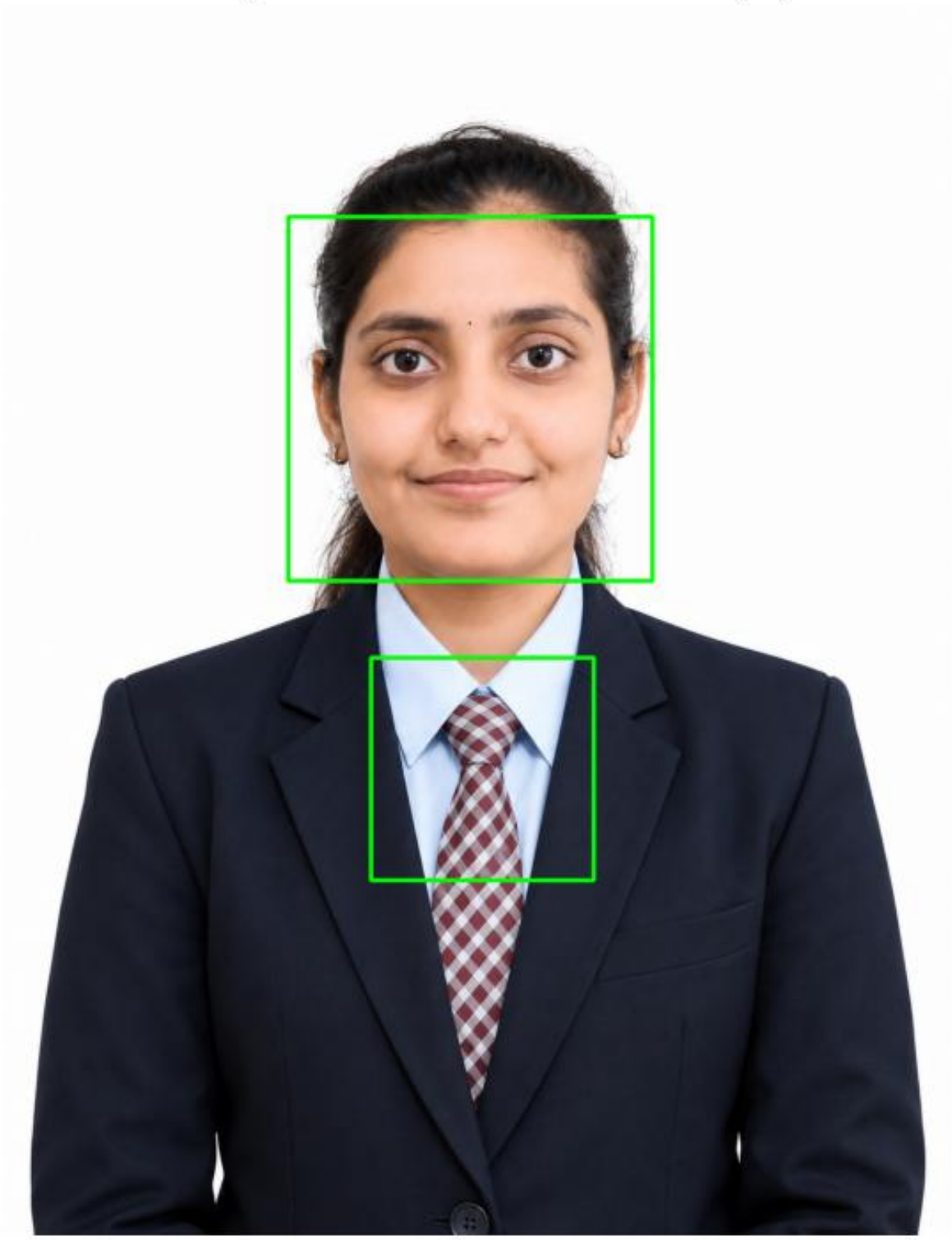


Figure 8. Haar Cascade detections on the portrait.

7.3 YOLO object detection

YOLO stands for You Only Look Once. It is a deep-learning-based object detector. Unlike the face-specific Haar Cascade, a pretrained YOLO model can detect multiple object categories represented in its training data. It analyses the input image and predicts bounding boxes, class labels and confidence scores.

The notebook loads the supplied yolo11n.pt weights and runs prediction on the portrait using a confidence threshold of 0.25. The confidence score expresses the model's confidence in a predicted detection; it should not be interpreted as a guarantee that the detection is correct.

YOLO class	Confidence reported
person	0.941070
tie	0.888866

The saved object-detection summary reports two YOLO boxes, with the classes person and tie. The person detection has confidence 0.941070 and the tie detection has confidence 0.888866. These are the classes and scores returned by the saved notebook run.



Figure 9. YOLO bounding boxes, labels and confidence scores.

7.4 Comparison of Haar Cascade and YOLO

Comparison point	Haar Cascade	YOLO
Detection purpose	Frontal face-like regions	Multiple trained object categories
Approach	Haar-like features and cascade classifiers	Deep-learning object detection
Notebook output	2 detections	2 detections: person and tie
Output information	Face-region bounding boxes	Bounding boxes, class names and confidence scores
Strength	Lightweight and specialised for frontal faces	Can detect different object categories in one image
Limitations	Sensitive to pose, lighting, scale and occlusion	Limited to trained classes and affected by confidence threshold, object size and occlusion

These detectors have different purposes. Haar Cascade is specialised for frontal faces, whereas YOLO is a general detector for supported classes. The fact that both returned two boxes does not mean they detected the same things or that their results can be directly compared by count alone.

7.5 Task 3 summary

The task demonstrates two approaches to object recognition. The Haar Cascade identifies face-like regions using a classical feature-based cascade. YOLO uses a pretrained deep-learning model to identify object classes and report confidence scores. The saved results show two Haar detections and two YOLO detections labelled person and tie.

8. Task 4 – Video Motion Estimation

8.1 Understanding video frames

A digital video is a sequence of still images called frames. When frames are displayed one after another at a suitable frame rate, they create the appearance of continuous movement. Motion estimation analyses differences between frames to determine which pixels or image features have changed position.

The notebook reads the supplied self-recorded video and reports its properties. The recorded video is 3840 × 2160 pixels, has a frame rate of 29.97 frames per second, contains 670 frames and has a duration of 22.36 seconds. The notebook processes 670 frames and records 23 sampled measurements. It uses a reduced working resolution during processing to make computation more manageable.

8.2 Frame differencing

Frame differencing is one of the simplest ways to identify movement. The method compares the current frame with the previous frame. If a pixel changes by more than a chosen threshold, it is treated as a possible moving pixel. The resulting binary mask highlights areas that changed between the two frames.

For example, if a person moves across a stationary camera view, the person's position changes from one frame to the next. The difference image may therefore highlight the person's old and new positions. Frame differencing is simple and fast, but it can also respond to camera shake, flickering lights, shadows and sudden illumination changes.

8.3 MOG2 background subtraction

MOG2 is a background subtraction method based on a Gaussian Mixture Model. It builds a statistical model of the background at each pixel over time. As new frames arrive, the model is updated. Pixels that do not fit the learned background model are marked as foreground.

The foreground mask can highlight moving people or objects while suppressing parts of the scene that remain relatively stable. Compared with simple frame differencing, MOG2 uses information from multiple frames to learn what is usually part of the background. However, it may still classify shadows, waving trees, moving screens or illumination changes as foreground.

8.4 Lucas–Kanade optical flow

Optical flow estimates the apparent movement of image points between consecutive frames. The Lucas–Kanade method tracks selected feature points by examining a small neighbourhood around each point. It assumes that the point's appearance remains sufficiently similar over a short time and that motion within the local neighbourhood can be approximated.

The notebook detects suitable points and attempts to track them from one frame to the next. Lines or arrows are drawn to show the estimated displacement. The direction indicates where a point appears to move, and the length represents the amount of displacement between the frames.

Lucas–Kanade provides point-level motion information rather than a complete foreground mask. It works best when the image contains clear corners or textured points. Points may be lost when the object moves too quickly, becomes hidden, leaves the frame or changes appearance.

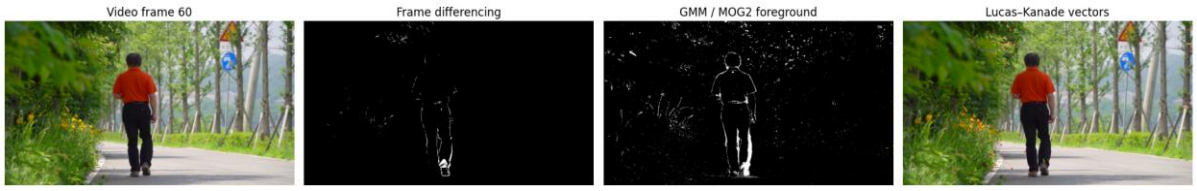


Figure 10. Representative video frame, frame-differencing mask, MOG2 foreground mask and Lucas–Kanade motion vectors.

8.5 Comparison of the three motion methods

Method	What it measures	Advantages	Limitations
Frame differencing	Pixel intensity changes between consecutive frames	Simple and fast	Sensitive to camera movement and lighting changes
MOG2	Foreground pixels that differ from a learned background model	Adapts to background over time	Shadows and dynamic backgrounds may be detected as foreground
Lucas–Kanade	Displacement of selected feature points	Shows direction and movement of tracked points	Requires trackable features and may lose points

The three methods provide different kinds of information. Frame differencing highlights changes between adjacent frames. MOG2 attempts to separate foreground from a learned background. Lucas–Kanade estimates the movement of selected points. They are complementary methods, and their outputs should be interpreted according to what each algorithm measures.

9. Overall Results and Discussion

- Image restoration: both filters improved the similarity measurements for the corresponding noisy image. The median-filtered result achieved the highest recorded SSIM of 0.9571.
- Segmentation: Otsu thresholding, morphology and Canny edge detection demonstrate how pixel regions and boundaries can be extracted. Their usefulness depends on image contrast and the choice of parameters.
- Watershed: the notebook includes the watershed implementation, but the saved run did not contain the optional dedicated multiple/touching-object image. The fallback output should therefore be treated as a demonstration of the code path rather than proof of successful touching-object separation.
- Feature extraction: HOG produced a descriptor of length 8,100. SIFT found 726 keypoints in image A and 489 in image B, with 74 good matches after the ratio test.
- Scale experiment: matching image B with its 75%-resized version produced 263 good matches, showing that many features remained detectable after resizing.
- Object detection: Haar Cascade reported two detections. YOLO reported two detections labelled person and tie.
- Motion estimation: the video was 22.36 seconds long and contained 670 frames. The notebook applied frame differencing, MOG2 and Lucas–Kanade to analyse motion.

10. Limitations and Considerations

- Image smoothing reduces noise but may also soften fine details.

- Global Otsu thresholding may not separate foreground and background when lighting or intensity distributions are complex.
- Morphological output depends on the structuring-element size and shape.
- Watershed segmentation depends heavily on good foreground and background markers and an appropriate image containing separable objects.
- HOG describes gradient patterns but does not provide semantic object labels.
- SIFT matching can contain incorrect correspondences; match visualisation and geometric verification are useful when accuracy is important.
- Haar Cascade is specialised for frontal-face patterns and may not detect faces at large angles or under occlusion.
- YOLO can only detect categories represented by its trained weights, and results depend on confidence settings and image quality.
- Motion estimation can be affected by camera movement, lighting variation, shadows, blur and objects moving too quickly.

11. Conclusion

This Computer Vision assignment implements a complete set of practical image and video analysis techniques using Python. The first task demonstrates how noise affects an image and how suitable filters can restore image quality. The recorded PSNR and SSIM values show that Gaussian filtering improved the Gaussian-noisy image, while median filtering produced a particularly strong result for salt-and-pepper noise. Thresholding, morphological operations, Canny edge detection and watershed introduce different ways to identify and refine image regions.

The second task demonstrates feature extraction and image matching. HOG represents local gradient structure, while SIFT identifies distinctive local keypoints and creates descriptors that can be compared between photographs. The saved two-photo experiment produced 74 good matches, and the resized-image experiment produced 263 good matches. These results illustrate how local features can be used to identify visual correspondences even when image scale changes.

The third task compares classical face detection with deep-learning object detection. Haar Cascade reported two face-like detections, while YOLO identified the person and tie classes with confidence scores. The comparison shows that different detection algorithms are designed for different purposes and should be selected according to the problem being solved.

The fourth task analyses movement in the supplied video. Frame differencing detects pixel changes between adjacent frames, MOG2 separates foreground from a learned background model, and Lucas–Kanade estimates the movement of selected feature points. Together, these methods demonstrate three different approaches to understanding motion in video.

Overall, the project provides practical experience with image enhancement, segmentation, feature description, object detection and motion analysis. It also demonstrates that algorithm selection, input quality and parameter settings strongly influence results. The outputs and numerical measurements in this documentation correspond to the saved notebook execution and should be updated if the notebook is rerun with different inputs or settings.

12. References

- OpenCV documentation – image processing, morphology, feature detection, object detection and video processing: https://docs.opencv.org/
- scikit-image documentation – image quality metrics and feature extraction: https://scikit-image.org/docs/

- Ultralytics documentation – YOLO model usage and prediction: https://docs.ultralytics.com/
- NumPy documentation – numerical arrays and random number generation: https://numpy.org/doc/